

BANCO DE DADOS

com Microsoft SQL Server

AULA 04

DML Parte I — INSERT

Inserindo dados reais no SISTEMA_VENDAS

INSERT básico

INSERT múltiplo

INSERT SELECT

IDENTITY

OUTPUT

SELECT INTO



Objetivos da Aula 04



Dominar a sintaxe completa do INSERT INTO



Usar OUTPUT para ver os valores inseridos automaticamente



Realizar inserções únicas e múltiplas em um único comando



Criar tabela e inserir dados com SELECT INTO



Copiar dados entre tabelas com INSERT...SELECT



Entender como constraints protegem durante a inserção



Entender IDENTITY e capturar o ID com SCOPE_IDENTITY()



Popular todas as 6 tabelas do SISTEMA_VENDAS



Revisão — Aula 03: Teoria dos Conjuntos

? O que retorna UNION?

✓ Todos os elementos de A e B, sem duplicatas (UNION ALL mantém duplicatas)

? O que retorna INTERSECT?

✓ Apenas os elementos presentes simultaneamente nas duas queries

? O que retorna EXCEPT?

✓ Elementos da 1ª query que NÃO estão na 2ª (A menos B)

? Por que NOT IN com NULL é perigoso?

✓ $\text{NULL IN (...) = UNKNOWN} \rightarrow$ a query retorna zero linhas inesperadamente

? Qual a regra obrigatória do UNION?

✓ Mesmo número de colunas e tipos compatíveis. ORDER BY só no último SELECT

DML — Data Manipulation Language

DML (Data Manipulation Language): comandos SQL que manipulam os DADOS dentro das tabelas. Diferente do DDL (que muda a estrutura), o DML lida com o conteúdo — inserir, consultar, atualizar e excluir registros.



INSERT

Aula 04

Adiciona novos registros a uma tabela



HOJE

Inserir dados reais nas 6 tabelas do SISTEMA_VENDAS



SELECT

Aula 05

Consulta e recupera dados existentes



UPDATE

Aula 08

Modifica valores em registros existentes



DELETE

Aula 08

Remove registros de uma tabela

INSERT INTO — Sintaxe e Variações

1. Inserção única

```
INSERT INTO tabela (col1, col2, col3)  
VALUES (val1, val2, val3);
```

2. Inserção múltipla

```
INSERT INTO tabela (col1, col2)  
VALUES (v1, v2), (v3, v4), (v5, v6);
```

3. INSERT...SELECT

```
INSERT INTO destino (col1, col2)  
SELECT col1, col2 FROM origem WHERE condição;
```

4. INSERT com OUTPUT

```
INSERT INTO tabela (col1)  
OUTPUT INSERTED.id, INSERTED.col1  
VALUES (val1);
```



Boa prática: SEMPRE especifique as colunas no INSERT — evita erros silenciosos se a tabela mudar de estrutura no futuro.



IDENTITY — Gerador Automático de PK

IDENTITY(seed, increment): gera automaticamente um valor único e sequencial para a PK. seed = valor inicial, increment = passo.
Exemplo: IDENTITY(1,1) começa em 1 e incrementa de 1 em 1.

```
SCOPE_IDENTITY()
```

✓ Recomendado

Retorna o último IDENTITY gerado na sessão e escopo atual. Seguro mesmo com triggers.

```
@@IDENTITY
```

⚠ Cuidado

Retorna o último IDENTITY gerado na sessão, incluindo triggers. Pode retornar ID inesperado.

```
IDENT_CURRENT('tabela')
```

i Informativo

Retorna o último IDENTITY gerado para uma tabela específica, por qualquer sessão.

```
DECLARE @novo_id INT; INSERT INTO CLIENTES (nome,estado) VALUES ('Teste','SP'); SET @novo_id =  
SCOPE_IDENTITY(); SELECT @novo_id;
```



Passo 1 — Populando CATEGORIAS

```
USE SISTEMA_VENDAS;
GO

-- =====
-- CATEGORIAS: inserir primeiro (não tem FK)
-- =====

INSERT INTO CATEGORIAS (nome_categoria, descricao, ativo)
VALUES

    ('Eletrônicos', 'Smartphones, notebooks, tablets e acessórios', 1),
    ('Roupas',      'Vestuário masculino, feminino e infantil',    1),
    ('Alimentos',   'Produtos alimentícios e bebidas variadas',    1),
    ('Móveis',      'Móveis para casa e escritório',              1),
    ('Esportes',    'Equipamentos e artigos esportivos',          1),
    ('Livros',      'Livros técnicos, didáticos e literatura',      1);

GO

-- Verificar inserção

SELECT id_categoria, nome_categoria, ativo
FROM   CATEGORIAS
ORDER BY id_categoria;

GO

-- Resultado esperado: 6 linhas com IDs 1 a 6
-- IDs gerados automaticamente pelo IDENTITY(1,1)

-- UNIQUE constraint em nome_categoria impede duplicatas
```



Passo 2 — Populando CLIENTES

```
-- =====
-- CLIENTES: 8 registros variados (diferentes estados)
-- =====

INSERT INTO CLIENTES (nome, email, telefone, cidade, estado)
VALUES

    ('Ana Silva',      'ana.silva@email.com',      '11999001001', 'São Paulo',      'SP'),
    ('Bruno Santos',   'bruno@email.com',      '21999002002', 'Rio de Janeiro', 'RJ'),
    ('Carla Oliveira', 'carla@email.com',      '31999003003', 'Belo Horizonte', 'MG'),
    ('Diego Ferreira', 'diego@email.com',      '41999004004', 'Curitiba',      'PR'),
    ('Eva Costa',      'eva@email.com',      '51999005005', 'Porto Alegre',   'RS'),
    ('Felipe Lima',    'felipe@email.com',    '11999006006', 'São Paulo',      'SP'),
    ('Gabriela Rocha', 'gabi@email.com',      '21999007007', 'Rio de Janeiro', 'RJ'),
    ('Henrique Dias',  NULL,                      '61999008008', 'Brasília',       'DF');

GO

-- Verificar: Henrique tem email NULL (campo opcional)
-- data_cadastro foi preenchida automaticamente pelo DEFAULT GETDATE()

SELECT id_cliente, nome, email, estado,
       CONVERT(DATE, data_cadastro) AS cadastrado_em
FROM   CLIENTES ORDER BY id_cliente;

GO
```




Passo 3 — Populando FUNCIONARIOS e PRODUTOS

```
-- ===== FUNCIONARIOS =====
INSERT INTO FUNCIONARIOS (nome, cargo, salario)
VALUES
    ('João Vendedor',      'Vendedor',          2800.00),
    ('Maria Gerente',      'Gerente',          5500.00),
    ('Pedro Estoque',      'Estoquista',       2200.00),
    ('Lucia Caixa',        'Operador Cx',      2000.00);

GO

-- ===== PRODUTOS (com id_categoria FK) =====
INSERT INTO PRODUTOS (nome_produto, preco, estoque, id_categoria)
VALUES
    ('iPhone 15 Pro',      6999.99,  15, 1), -- Eletrônicos
    ('Notebook Dell i7',   3499.00,   8, 1), -- Eletrônicos
    ('Fone Bluetooth JBL', 199.90,  50, 1), -- Eletrônicos
    ('Camisa Polo Pq',      89.90, 100, 2), -- Roupas
    ('Calça Jeans 42',     149.90,  60, 2), -- Roupas
    ('Arroz Integral 5kg',  28.90, 200, 3), -- Alimentos
    ('Café Gourmet 500g',  42.50, 150, 3), -- Alimentos
    ('Sofá 3 Lugares',     1890.00,   5, 4), -- Móveis
    ('Bicicleta 21v',      890.00,  12, 5), -- Esportes
    ('Tênis Running Pro',  299.90,  35, 5); -- Esportes

GO
```



Passo 4 — Populando PEDIDOS e ITENS_PEDIDO

```
-- ===== PEDIDOS (FK: id_cliente, id_funcionario) =====
INSERT INTO PEDIDOS (id_cliente, id_funcionario, status, total)
VALUES
    (1, 1, 'ENTREGUE', 479.80), -- Ana comprou com João
    (2, 1, 'ENVIADO', 7199.89), -- Bruno comprou com João
    (3, 2, 'APROVADO', 268.70), -- Carla comprou com Maria
    (1, 3, 'PENDENTE', 42.50), -- Ana comprou com Pedro
    (5, 1, 'CANCELADO', 89.90), -- Eva cancelou
    (6, 2, 'ENTREGUE', 1890.00); -- Felipe comprou com Maria

GO

-- ===== ITENS_PEDIDO =====
INSERT INTO ITENS_PEDIDO (id_pedido, id_produto, quantidade, preco_unitario)
VALUES
    (1, 4, 2, 89.90), -- Pedido 1: 2x Camisa Polo = 179.80
    (1, 10, 1, 299.90), -- Pedido 1: 1x Tênis Running = 299.90
    (2, 1, 1, 6999.99), -- Pedido 2: 1x iPhone 15 = 6999.99
    (2, 3, 1, 199.90), -- Pedido 2: 1x Fone BT = 199.90
    (3, 5, 1, 149.90), -- Pedido 3: 1x Calça Jeans = 149.90
    (3, 6, 2, 28.90), -- Pedido 3: 2x Arroz = 57.80
    (3, 7, 1, 42.50), -- Pedido 3: 1x Café = 42.50
    (4, 7, 1, 42.50), -- Pedido 4: 1x Café = 42.50
    (5, 4, 1, 89.90), -- Pedido 5: 1x Camisa (canc)= 89.90
    (6, 8, 1, 1890.00); -- Pedido 6: 1x Sofá = 1890.00
```

GO



Verificando os Dados Inseridos

```
-- ===== CONTAGEM DE REGISTROS POR TABELA =====  
  
SELECT 'CATEGORIAS' AS tabela, COUNT(*) AS registros FROM CATEGORIAS  
UNION ALL  
SELECT 'CLIENTES', COUNT(*) FROM CLIENTES  
UNION ALL  
SELECT 'FUNCIONARIOS', COUNT(*) FROM FUNCIONARIOS  
UNION ALL  
SELECT 'PRODUTOS', COUNT(*) FROM PRODUTOS  
UNION ALL  
SELECT 'PEDIDOS', COUNT(*) FROM PEDIDOS  
UNION ALL  
SELECT 'ITENS_PEDIDO', COUNT(*) FROM ITENS_PEDIDO;  
GO  
  
-- Resultado esperado:  
  
-- CATEGORIAS → 6 registros  
-- CLIENTES → 8 registros  
-- FUNCIONARIOS → 4 registros  
-- PRODUTOS → 10 registros  
-- PEDIDOS → 6 registros  
-- ITENS_PEDIDO → 10 registros  
  
-- Se os números batem: SISTEMA_VENDAS populado com sucesso! 
```



SCOPE_IDENTITY() e OUTPUT na Prática

```
-- ===== CAPTURAR ID GERADO: SCOPE_IDENTITY() =====
-- Cenário: criar um pedido e seus itens em sequência

DECLARE @id_pedido INT;

-- Inserir o pedido
INSERT INTO PEDIDOS (id_cliente, id_funcionario, status, total)
VALUES (4, 1, 'PENDENTE', 0.00);

-- Capturar o ID gerado
SET @id_pedido = SCOPE_IDENTITY();
PRINT 'Novo pedido criado com ID: ' + CAST(@id_pedido AS VARCHAR);

-- Usar o ID capturado para inserir os itens
INSERT INTO ITENS_PEDIDO (id_pedido, id_produto, quantidade, preco_unitario)
VALUES (@id_pedido, 9, 1, 890.00),      -- Bicicleta 21v
       (@id_pedido, 5, 2, 149.90);      -- 2x Calça Jeans

GO

-- ===== OUTPUT: ver dados inseridos em tempo real ===
INSERT INTO CLIENTES (nome, cidade, estado)
OUTPUT INSERTED.id_cliente, INSERTED.nome, INSERTED.data_cadastro
VALUES ('Isabela Nunes', 'Fortaleza', 'CE');

-- OUTPUT retorna os dados inseridos SEM precisar de outro SELECT!
```



INSERT...SELECT e SELECT INTO

```
-- ===== INSERT...SELECT: copiar dados entre tabelas ==
-- Cenário: copiar clientes de SP para tabela de campanha

-- Passo 1: criar tabela destino (se não existir)
CREATE TABLE CAMPANHA_SP (
    id_campanha INT IDENTITY(1,1) PRIMARY KEY,
    nome        VARCHAR(150),
    email       VARCHAR(200),
    importado_em DATETIME2 DEFAULT GETDATE()
);
GO

-- Passo 2: INSERT...SELECT para popular a tabela
INSERT INTO CAMPANHA_SP (nome, email)
SELECT nome, email FROM CLIENTES
WHERE estado = 'SP' AND email IS NOT NULL;
GO

-- ===== SELECT INTO: criar + inserir em um comando! =
-- Não precisa criar a tabela antes
SELECT nome, email, cidade, estado,
       GETDATE() AS data_backup
INTO   CLIENTES_BACKUP_2025 -- tabela criada automaticamente!
FROM   CLIENTES;
GO
```

SET IDENTITY_INSERT — Controle Manual do IDENTITY

Por padrão, não é possível inserir um valor específico em uma coluna IDENTITY — o SQL Server gera o valor automaticamente. O SET IDENTITY_INSERT permite FORÇAR um valor específico.



Migração de dados

Importar dados de outro sistema mantendo os IDs originais



Restore seletivo

Restaurar registros específicos com seus IDs originais



Ambiente de testes

Criar dados de teste com IDs previsíveis para automação



Uso em produção rotineiro

NÃO use em rotina — causa gaps no IDENTITY e possível conflito

```
SET IDENTITY_INSERT CLIENTES ON; -- Habilitar inserção manual
INSERT INTO CLIENTES (id_cliente, nome, estado) -- id_cliente obrigatório!
VALUES (999, 'Cliente Migrado', 'SP');
SET IDENTITY_INSERT CLIENTES OFF; -- SEMPRE desabilitar depois!
DBCC CHECKIDENT('CLIENTES', RESEED, 999); -- Reajustar o IDENTITY
```



Boas Práticas de Inserção de Dados

01



Sempre liste as colunas

INSERT INTO tabela (col1, col2) VALUES (...) — nunca omita as colunas. Se a tabela mudar, o código não quebra silenciosamente.

02



Use transações para inserções relacionadas

BEGIN TRANSACTION → INSERT no pedido → INSERT nos itens → COMMIT. Se um falhar, ROLLBACK reverte tudo.

03



Prefira INSERT múltiplo

VALUES (v1),(v2),(v3) é muito mais eficiente que 3 INSERTs separados. Reduz round-trips ao banco de dados.

04



Use SCOPE_IDENTITY() para capturar IDs

Nunca use MAX(id) ou @@IDENTITY para capturar o ID recém-inserido em ambientes multi-usuário.

05



Valide os dados antes de inserir

Use CHECK constraints no banco E valide na aplicação. A defesa em camadas evita dados inválidos mesmo em acesso direto.



Exercícios Propostos — Aula 04

01

Básico

Insira 3 novos clientes de cidades diferentes. Após cada inserção, use `SCOPE_IDENTITY()` para capturar e exibir o ID gerado. Verifique com `SELECT`.

02

Básico

Insira 2 novos produtos: um livro técnico (categoria 6) e um smartwatch (categoria 1). Use `INSERT` múltiplo em um único comando.

03

Médio

Crie um novo pedido para o cliente 'Diego Ferreira' com 2 itens. Use `DECLARE + SCOPE_IDENTITY()` para capturar o `id_pedido` e criar os `ITENS_PEDIDO` com esse ID.

04

Médio

Use `SELECT INTO` para criar uma tabela `PRODUTOS_ELETRONICOS` com todos os produtos da categoria Eletrônicos (`id_categoria=1`). Verifique o resultado.

05

Avançado

Crie um script completo com transação: `BEGIN TRANSACTION` → insira 1 pedido → insira seus itens → calcule o total com subquery → atualize o campo total do pedido → `COMMIT`. Inclua `TRY/CATCH` com `ROLLBACK` em caso de erro.



Respostas Comentadas — Exercícios 03 e 04

```
-- EX 03: Pedido com SCOPE_IDENTITY()
DECLARE @id_pedido INT;

INSERT INTO PEDIDOS (id_cliente, id_funcionario, status, total)
VALUES (4, 1, 'PENDENTE', 0.00); -- Diego Ferreira / João

SET @id_pedido = SCOPE_IDENTITY();
PRINT 'Pedido #' + CAST(@id_pedido AS VARCHAR) + ' criado!';

INSERT INTO ITENS_PEDIDO (id_pedido, id_produto, quantidade, preco_unitario)
VALUES (@id_pedido, 2, 1, 3499.00), -- Notebook Dell
       (@id_pedido, 3, 1, 199.90); -- Fone Bluetooth
GO

-- EX 04: SELECT INTO para Eletrônicos
SELECT p.id_produto, p.nome_produto, p.preco, p.estoque,
       c.nome_categoria,
       GETDATE() AS exportado_em
INTO PRODUTOS_ELETRONICOS -- tabela criada automaticamente!
FROM PRODUTOS p
JOIN CATEGORIAS c ON p.id_categoria = c.id_categoria
WHERE p.id_categoria = 1; -- apenas Eletrônicos
GO
```



Desafio da Aula 04

 Contexto: A empresa abriu uma filial em Salvador (BA) e contratou 2 novos vendedores. Chegaram novos produtos e os primeiros pedidos da nova filial precisam ser registrados.

Tarefas:

1. Insira 2 novos funcionários com cargo 'Vendedor' e salário R\$ 2.600
2. Insira 3 novos clientes de Salvador/BA com emails válidos
3. Insira 2 novos produtos: 'Cadeira Gamer' (Móveis, R\$899) e 'Mouse Sem Fio' (Eletrônicos, R\$129)
4. Crie 2 pedidos para clientes de Salvador usando DECLARE + SCOPE_IDENTITY()
5. Para cada pedido, insira pelo menos 2 itens usando o ID capturado
6. Crie uma tabela CLIENTES_SALVADOR com SELECT INTO contendo apenas os clientes da BA
7. BÔNUS: Use INSERT...SELECT para copiar os produtos de Eletrônicos para uma tabela CATALOGO_DIGITAL com os campos: nome_produto, preco, 'Disponível Online' AS status



Resumo — Aula 04: INSERT



INSERT básico

INSERT INTO tabela (colunas) VALUES (valores) — sempre especifique as colunas



INSERT...SELECT

Copia dados de outra tabela — ideal para migração e carga



OUTPUT

INSERTED.coluna retorna dados inseridos sem precisar de outro SELECT



Constraints na inserção

FK, CHECK e UNIQUE rejeitam dados inválidos automaticamente — o banco se protege



INSERT múltiplo

VALUES (v1),(v2),(v3) — um único comando, muito mais eficiente



SCOPE_IDENTITY()

Captura o ID gerado pelo INSERT. Use sempre este, nunca @@IDENTITY



SELECT INTO

Cria nova tabela e insere dados em um único comando — ótimo para backups



Ordem de inserção

Tabelas sem FK primeiro (CATEGORIAS → PRODUTOS, CLIENTES → PEDIDOS → ITENS)



Próxima Aula 05: SELECT e WHERE

Aula 05 — O que vem por aí:



SELECT completo

Estrutura, cláusulas e ordem de execução



WHERE

Filtrar registros com operadores relacionais



AND / OR / NOT

Combinar condições — precedência e parênteses



DISTINCT e Alias

Eliminar duplicatas e renomear colunas



ORDER BY

Ordenar resultados em ASC e DESC



TOP e OFFSET/FETCH

Limitar e paginar resultados



Para a próxima aula: conclua o desafio e garanta que o SISTEMA_VENDAS tem todos os dados inseridos corretamente!